

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE CODE:** MLA0304

**COURSE NAME:** Reinforcement learning for Environment and Agent

---

### COURSE OUTCOME COVERED

**CO3:** *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

*Assessment Tool Weightage: 30%*

---

Dr DUMPALA PRASANTH

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

### CODING CHALLENGE

**Duration: 2hrs**

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as  $\epsilon$ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor ( $\gamma$ ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

---

***Code of Conduct:***

*I (V Sunil Kumar /192425292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

V Sunil Kumar

## RUBRICS FOR CALCULATION

| Criteria              | Weightage | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                          | Level 2<br>(Developing)                            | Level 1<br>(Beginning)                |
|-----------------------|-----------|-----------------------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------|---------------------------------------|
| Technical Knowledge   | 25        | Demonstrates strong and accurate subject knowledge with in-depth understanding    | Shows good knowledge with minor gaps             | Basic knowledge with noticeable errors             | Lacks fundamental technical knowledge |
| Problem Solving       | 25        | Solves problems logically and applies concepts effectively in real scenarios      | Applies concepts with moderate accuracy          | Limited problem-solving ability; requires guidance | Unable to solve or apply concepts     |
| Analytical Thinking   | 15        | Analyzes questions critically and provides well-structured, logical answers       | Provides reasonable analysis with some structure | Superficial or partially structured responses      | No analytical thinking demonstrated   |
| Communication         | 15        | Communicates clearly, confidently, and professionally with proper terminology     | Communicates well with minor hesitation          | Communication lacks clarity or confidence          | Unable to communicate effectively     |
| Confidence            | 15        | Displays high confidence, positive attitude, and professional behavior throughout | Shows good confidence with minor issues          | Low confidence; inconsistent behavior              | Lacks confidence and professionalism  |
| Response to Questions | 10        | Responds accurately, promptly, and elaborates with relevant examples              | Responds correctly but with limited depth        | Partial responses; needs prompting                 | Unable to respond appropriately       |

# 1. Reinforcement Learning Framework

Reinforcement Learning (RL) is a machine learning approach in which an agent learns by interacting with an environment and receiving rewards or penalties.

Agent–Environment Interaction

The interaction works as follows:

Agent  $\rightarrow$  Action  $\rightarrow$  Environment  $\rightarrow$  Reward + Next State  $\rightarrow$  Agent

- Agent: The learner or decision-maker.
- Environment: The world in which the agent operates.
- State (S): Current situation of the environment.
- Action (A): Decision taken by the agent.
- Reward (R): Feedback received after taking an action.
- Policy ( $\pi$ ): Strategy used by the agent to select actions.

Example

For a robot navigating a warehouse:

- State = Robot's current position.
- Action = Move left, right, forward, or backward.
- Reward = +10 for reaching the destination.
- Penalty = -10 for hitting an obstacle.

Cumulative Reward

The agent does not only consider the immediate reward. It tries to maximize the total future reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where  $\gamma$  is the discount factor.

Thus, RL helps the agent learn decisions that provide the maximum long-term benefit.

## Code

```
import random
```

```
states = ["Start", "Middle", "Goal"]  
actions = ["Move", "Stay"]
```

```
state = "Start"  
total_reward = 0
```

```

for step in range(5):
    action = random.choice(actions)

    if state == "Start" and action == "Move":
        next_state = "Middle"
        reward = 1
    elif state == "Middle" and action == "Move":
        next_state = "Goal"
        reward = 10
    elif state == "Goal":
        next_state = "Goal"
        reward = 0
    else:
        next_state = state
        reward = -1

    print("State:", state,
          "Action:", action,
          "Reward:", reward,
          "Next State:", next_state)

    total_reward += reward
    state = next_state

print("Total Reward:", total_reward)

```

### Sample Output

```

State: Start Action: Move Reward: 1 Next State: Middle
State: Middle Action: Move Reward: 10 Next State: Goal
State: Goal Action: Stay Reward: 0 Next State: Goal
State: Goal Action: Move Reward: 0 Next State: Goal
State: Goal Action: Stay Reward: 0 Next State: Goal
Total Reward: 11

```

**Concept demonstrated:** State → Action → Reward → Next State.

## 2. Decision-Making as a Markov Decision Process

A **Markov Decision Process (MDP)** provides a mathematical framework for representing decision-making problems in RL.

An MDP is represented as:

$$\text{MDP} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

### Components

1. **State Space (S)**  
Set of all possible states of the environment.
2. **Action Space (A)**  
Set of actions available to the agent.

Example:

$$\mathcal{A} = \{\text{Left}, \text{Right}, \text{Up}, \text{Down}\}$$

3. **Transition Probability P**

It represents the probability of moving from one state to another after performing an action.

$$P(s'|s, a)$$

where:

- $s$  = current state
  - $a$  = action
  - $s'$  = next state
4. **Reward Function R**

Defines the reward received for taking an action.

$$R(s, a, s')$$

5. **Discount Factor  $\gamma$**

Determines the importance of future rewards.

$$0 \leq \gamma \leq 1$$

- $\gamma$  close to 0  $\rightarrow$  focuses on immediate rewards.
- $\gamma$  close to 1  $\rightarrow$  considers future rewards strongly.

### Example

In a robot navigation problem, the MDP helps the robot decide which movement will maximize its expected future reward.

## Code

```
states = ["A", "B", "C"]
actions = ["Left", "Right"]

transition_probability = {
    ("A", "Right"): {"B": 1.0},
    ("B", "Right"): {"C": 1.0},
    ("B", "Left"): {"A": 1.0}
}

rewards = {
    ("A", "Right"): 1,
    ("B", "Right"): 10,
    ("B", "Left"): -1
}

gamma = 0.9

print("States:", states)
print("Actions:", actions)
print("Discount Factor:", gamma)

for key in transition_probability:
    print("\nState:", key[0])
    print("Action:", key[1])
    print("Transition:", transition_probability[key])
    print("Reward:", rewards[key])
```

## Sample Output

```
States: ['A', 'B', 'C']
Actions: ['Left', 'Right']
Discount Factor: 0.9
```

```
State: A
Action: Right
Transition: {'B': 1.0}
Reward: 1
```

```
State: B
Action: Right
Transition: {'C': 1.0}
Reward: 10
```

```
State: B
Action: Left
Transition: {'A': 1.0}
Reward: -1
```

**Concept demonstrated:** MDP = States + Actions + Transition Probability + Reward + Discount Factor.

### 3. Policy and Value Functions

A **policy** defines how an RL agent selects actions.

$$\pi(a|s) = P(A_t = a | S_t = s)$$

It can be deterministic or probabilistic.

#### Value Functions

Value functions estimate the **long-term usefulness of states and actions**.

#### State-Value Function

The state-value function  $V_\pi(s)$  represents the expected return when the agent starts from state  $s$  and follows policy  $\pi$ .

$$V_\pi(s) = E_\pi[G_t | S_t = s]$$

#### Action-Value Function

The action-value function  $Q_\pi(s, a)$  represents the expected return after taking action  $a$  in state  $s$ .

$$Q_\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$$

#### Bellman Equation

The Bellman equation breaks the value of a state into:

#### Immediate reward + discounted future value

For state value:

$$V_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_\pi(s')]$$

For Q-value:

$$Q(s, a) = R + \gamma a' \max_{a'} Q(s', a')$$

Thus, value functions help the agent evaluate **which states and actions provide better long-term rewards**.

#### Code

gamma = 0.9

reward = 10



```
future_value = 5

value = reward + gamma * future_value

print("Immediate Reward:", reward)
print("Future Value:", future_value)
print("Discount Factor:", gamma)
print("Bellman Value:", value)
```

## Output

```
Immediate Reward: 10
Future Value: 5
Discount Factor: 0.9
Bellman Value: 14.5
```

The Bellman equation is:

$$V(s) = R + \gamma V(s')$$

Therefore:

$$V(s) = 10 + (0.9)(5) = 14.5$$

## 4. Exploration–Exploitation Trade-off

An RL agent faces two choices:

### Exploration

Trying new actions to discover potentially better rewards.

### Exploitation

Choosing the action that currently appears to give the highest reward.

If an agent only exploits, it may never discover a better strategy. If it only explores, it may waste time taking poor actions.

Therefore, RL requires a balance between **exploration and exploitation**.

### $\epsilon$ -Greedy Strategy

With probability  $\epsilon$ , the agent chooses a random action.

With probability  $1-\epsilon$ , it chooses the best-known action.

$a = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg\max_a Q(s,a) & \text{with probability } 1-\epsilon \end{cases}$

Example:

If  $\epsilon = 0.1$ :

- 10%  $\rightarrow$  exploration
- 90%  $\rightarrow$  exploitation

### Softmax Strategy

Softmax selects actions according to their Q-values:

$$P(a|s) = \frac{e^{Q(s,a)}}{\sum_a e^{Q(s,a)}}$$

where  $\tau$  is the **temperature parameter**.

- High  $\tau \rightarrow$  more exploration.
- Low  $\tau \rightarrow$  more exploitation.

### Comparison

#### $\epsilon$ -Greedy

Simple to implement

#### Softmax

More gradual action selection

Random exploration

Probability-based exploration

Does not distinguish between bad actions

Considers Q-value differences

Computationally cheap

Slightly more computationally expensive

Code

```
import random
import math
```

```
q_values = [2, 5, 3]
epsilon = 0.2
```

```
# Epsilon-Greedy
if random.random() < epsilon:
    action = random.randint(0, 2)
    method = "Exploration"
else:
    action = q_values.index(max(q_values))
    method = "Exploitation"
```

```
print("Q-values:", q_values)
print("Epsilon:", epsilon)
print("Selected Action:", action)
print("Method:", method)
```

```
# Softmax
temperature = 1.0
```

```
exp_values = [math.exp(q / temperature) for q in q_values]
total = sum(exp_values)
```

```
probabilities = [x / total for x in exp_values]
```

```
print("\nSoftmax Probabilities:")
for i, p in enumerate(probabilities):
    print("Action", i, ":", round(p, 3))
```

Sample Output

Q-values: [2, 5, 3]

Epsilon: 0.2

Selected Action: 1

Method: Exploitation

Softmax Probabilities:

Action 0 : 0.042

Action 1 : 0.844

Action 2 : 0.114

Concept:  $\epsilon$ -greedy uses random exploration, while Softmax assigns probabilities based on Q-values.

## 5. Reward Shaping in Reinforcement Learning

**Reward shaping** means providing additional rewards or penalties to guide the agent toward desirable behavior.

Instead of giving a reward only when the final goal is achieved, intermediate actions can also receive rewards.

### Example

Consider a robot navigating to a destination.

Without reward shaping:

- Reaching goal = +100
- Other actions = 0

With reward shaping:

- Moving closer to goal = +5
- Moving away = -2
- Hitting obstacle = -10
- Reaching goal = +100

### Benefits

1. Provides more frequent feedback.
2. Accelerates learning.
3. Helps the agent discover useful actions.
4. Reduces the time required to reach the goal.
5. Can prevent the agent from repeatedly selecting poor actions.

### Important Point

Poorly designed rewards can create **unintended behavior**.

For example, if a robot receives rewards simply for moving, it may keep moving without reaching the destination.

Therefore, reward shaping should encourage the **desired final behavior**, not just individual actions.

### Code

```
positions = [0, 1, 2, 3, 4]
goal = 4
```

```

total_reward = 0

for position in positions[:-1]:
    next_position = position + 1

    if next_position == goal:
        reward = 10
    elif next_position > position:
        reward = 1
    else:
        reward = -1

    print("Position:", position,
          "Next Position:", next_position,
          "Reward:", reward)

    total_reward += reward

print("Total Reward:", total_reward)

```

Output

```

Position: 0 Next Position: 1 Reward: 1
Position: 1 Next Position: 2 Reward: 1
Position: 2 Next Position: 3 Reward: 1
Position: 3 Next Position: 4 Reward: 10
Total Reward: 13

```

Here, the agent receives small rewards for moving toward the goal and a larger reward for reaching it.

## 6. Challenges of RL in Real-World Robotics

Applying RL to real robots is more difficult than applying it in simulations.

### 1. Safety

Incorrect actions can damage the robot or its surroundings.

Example:

A robot may collide with a person or expensive equipment.

Therefore, safety constraints are necessary.

### 2. Sample Efficiency

RL often requires a large number of interactions to learn a good policy.

Real robots operate slowly and collecting millions of samples can be expensive and time-consuming.

### 3. Environment Variability

Real-world environments constantly change.

Examples:

- Lighting changes.
- Objects move.
- Sensors produce noise.
- People move around.
- Floor conditions change.

A policy learned in one environment may not work well in another.

### 4. Hardware Limitations

Robots have limitations such as:

- Battery power
- Sensor accuracy
- Processing capacity
- Mechanical wear

### Solutions

Simulation, transfer learning, safe exploration, and domain randomization can reduce these problems.

Code

```

grid = [
    ["S", ".", "."],
    [".", "X", "."],
    [".", ".", "G"]
]

position = [0, 0]

moves = [
    (1, 0), # Down
    (0, 1), # Right
]

total_reward = 0

for move in moves:
    new_row = position[0] + move[0]
    new_col = position[1] + move[1]

    if grid[new_row][new_col] == "X":
        reward = -10
        print("Obstacle detected! Reward:", reward)
        break

    position = [new_row, new_col]

    if grid[new_row][new_col] == "G":
        reward = 10
        print("Goal reached! Reward:", reward)
    else:
        reward = 1
        print("Safe movement. Reward:", reward)

    total_reward += reward

print("Total Reward:", total_reward)

```

Sample Output

Safe movement. Reward: 1

Safe movement. Reward: 1

Total Reward: 2

Concept: A robot must learn while avoiding dangerous states such as obstacles.

## 7. Influence of Discount Factor $\gamma$

The **discount factor**  $\gamma$  determines how much importance the agent gives to future rewards.

The return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where:

$$0 \leq \gamma \leq 1$$

### Low $\gamma$

When  $\gamma$  is close to 0, the agent focuses mainly on immediate rewards.

Example:

$$\gamma = 0.1$$

The agent behaves more **short-term oriented**.

### High $\gamma$

When  $\gamma$  is close to 1, future rewards become more important.

Example:

$$\gamma = 0.9$$

The agent considers **long-term consequences**.

### Example

Suppose an agent can:

- Take an immediate reward of 5.
- Take another action that gives 20 after several steps.

With a low  $\gamma$ , the agent may choose 5.

With a high  $\gamma$ , it may choose the action leading to 20.

### Episodic Tasks

In episodic tasks such as games or maze solving, the agent eventually reaches a terminal state.

A suitable  $\gamma$  helps the agent balance immediate rewards with the final goal.

Therefore,  $\gamma$  strongly influences the agent's **planning horizon and behavior**.



## Code

```
reward_now = 5  
future_reward = 20  
  
for gamma in [0.1, 0.5, 0.9]:  
    total_value = reward_now + gamma * future_reward  
  
    print("Gamma:", gamma,  
          "Value:", total_value)
```

## Output

Gamma: 0.1 Value: 7.0

Gamma: 0.5 Value: 15.0

Gamma: 0.9 Value: 23.0

## Interpretation

Low gamma → Focuses on immediate rewards

High gamma → Gives more importance to future rewards

## 8. Model-Free vs Model-Based Reinforcement Learning

### Model-Free RL

The agent learns directly from experience without explicitly learning the environment's transition model.

Examples:

- Q-Learning
- SARSA
- Policy Gradient

### Advantages

- Simple conceptual structure.
- Works well in complex environments.
- Does not require an accurate environment model.

### Limitations

- Requires many interactions.
  - Can be sample inefficient.
  - Learning can be slow.
- 

### Model-Based RL

The agent learns or uses a model of the environment.

The model estimates:

$P(s'|s,a)$

and the expected rewards.

The agent can use this model to predict future states before taking actions.

### Advantages

- More sample efficient.
- Can plan ahead.
- Useful when interactions are expensive.

### Limitations

- Learning an accurate model can be difficult.
- Model errors can lead to poor decisions.
- More computationally complex.

## Comparison

| Model-Free                      | Model-Based                     |
|---------------------------------|---------------------------------|
| Learns directly from experience | Learns/uses environment model   |
| Usually sample inefficient      | More sample efficient           |
| No planning model required      | Supports planning               |
| Easier to implement             | More complex                    |
| Good for complex environments   | Good when samples are expensive |

In highly dynamic environments, model-free methods can be robust, while model-based methods can be advantageous when accurate prediction and planning are possible.

## Code

```
# Model-Free example
```

```
q_value = 5
```

```
reward = 10
```

```
learning_rate = 0.5
```

```
q_value = q_value + learning_rate * (reward - q_value)
```

```
print("Model-Free Q-value:", q_value)
```

```
# Model-Based example
```

```
transition_model = {
```

```
    "A": "B",
```

```
    "B": "C"
```

```
}
```

```
current_state = "A"
```

```
next_state = transition_model[current_state]
```

```
print("Model-Based Prediction:")
```

```
print("Current State:", current_state)
```

```
print("Predicted Next State:", next_state)
```

### **Output**

Model-Free Q-value: 7.5

Model-Based Prediction:

Current State: A

Predicted Next State: B

### **Concept:**

- Model-free → learns from experience.
- Model-based → uses a model to predict future states.

## 9. Q-Learning Update Example

**Q-learning** is an off-policy, model-free RL algorithm that learns the optimal action-value function.

The update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- $\alpha$  = learning rate
- $\gamma$  = discount factor
- $R$  = reward
- $s'$  = next state

### Example

Assume:

$$Q(s,a)=5 \quad \alpha=0.5, \gamma=0.9$$

The agent receives:

$$R=10$$

and the maximum Q-value of the next state is:

$$\max Q(s',a')=8$$

First calculate the target:

$$\text{Target} = 10 + (0.9)(8) \quad \text{Target} = 17.2$$

Now update:

$$Q_{\text{new}} = 5 + 0.5(17.2 - 5) \quad Q_{\text{new}} = 11.1$$

Therefore, the Q-value changes from **5** to **11.1**.

### Convergence

As the agent repeatedly interacts with the environment, Q-values are updated.

Initially:

**Q-values are inaccurate** → **More experience** → **Better Q-values** → **Stable Q-values**

Under suitable conditions, such as sufficient exploration and appropriate learning rates, Q-learning can converge toward the **optimal Q-function**.

## Code

```
alpha = 0.5  
gamma = 0.9  
  
q_value = 5  
reward = 10  
next_max_q = 8  
  
print("Initial Q-value:", q_value)  
  
for episode in range(5):  
  
    target = reward + gamma * next_max_q  
  
    q_value = q_value + alpha * (target - q_value)  
  
    print("Episode", episode + 1,  
          "Q-value:", round(q_value, 2))
```

## Output

Initial Q-value: 5

Episode 1 Q-value: 11.1

Episode 2 Q-value: 14.15

Episode 3 Q-value: 15.68

Episode 4 Q-value: 16.44

Episode 5 Q-value: 16.82

The target is:

$10 + (0.9 \times 8) = 17.2$

The Q-value gradually approaches **17.2**.

This demonstrates **convergence**.

### **Q-Learning Formula**

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

## 10. On-Policy vs Off-Policy Learning

The major difference is the policy used for **learning** and **behavior**.

### On-Policy Learning

The agent learns about the same policy that it uses to select actions.

Example:

#### SARSA

SARSA update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma Q(s',a') - Q(s,a)]$$

The next action  $a'$  is selected according to the current behavior policy.

### Off-Policy Learning

The agent can behave using one policy but learn about another policy.

Example:

#### Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

It learns the greedy optimal policy even if the agent explores using another behavior policy.

### Comparison

| On-Policy                          | Off-Policy                               |
|------------------------------------|------------------------------------------|
| Learns the policy being followed   | Learns a different target policy         |
| SARSA is an example                | Q-learning is an example                 |
| Generally more conservative        | Can learn optimal policy while exploring |
| Exploration affects learned policy | Separates behavior and target policies   |



Useful when actual behavior matters

Useful when learning an optimal policy from exploratory data

### Code

```
alpha = 0.5
```

```
gamma = 0.9
```

```
reward = 10
```

```
# Initial values
```

```
q_sarsa = 5
```

```
q_learning = 5
```

```
next_q_sarsa = 6
```

```
next_max_q = 8
```

```
# SARSA update
```

```
q_sarsa = q_sarsa + alpha * (  
    reward + gamma * next_q_sarsa - q_sarsa  
)
```

```
# Q-learning update
```

```
q_learning = q_learning + alpha * (
```

```

        reward + gamma * next_max_q - q_learning
    )

print("SARSA (On-Policy) Q-value:", round(q_sarsa,
2))

print("Q-Learning (Off-Policy) Q-value:",
round(q_learning, 2))

```

### **Output**

SARSA (On-Policy) Q-value: 10.2

Q-Learning (Off-Policy) Q-value: 11.1

### **Why are they different?**

#### **SARSA:**

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma Q(s',a') - Q(s,a)]$$

It uses the **actual next action**.

#### **Q-Learning:**

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$